PROJECT DOCUMENTATION

Multi-Line Insurance Policy and Claims Management System

A Salesforce solution for Vehicle, Property and Life insurance: guided quoting, premium calculation, claim routing, approvals and an adjuster dashboard in one place.

TEAM ID	SWTID-2026-8095
PLATFORM	Salesforce
ORG TYPE	Developer Edition
AUTOMATION	Flow, Approval, Apex, LWC
TEAM SIZE	4 members

OUR TEAM

S. Nithish TEAM LEADER
NM ID: 13B27FAE19C56F09F3543B6453644851
Register No: 821923243037

G. Anish Pradeep TEAM MEMBER
NM ID: E588806BA58D395989045D185F8FC289
Register No: 821923243007

S. Johnson Infant TEAM MEMBER
NM ID: B79E6A8BC3205E4B2EA407B0225A3E24
Register No: 821923243023

S. Bharathraj TEAM MEMBER
NM ID: 55E53093039601EBC7E49C15B5D3CF60
Register No: 821923243012

ST. JOSEPH'S COLLEGE OF ENGINEERING AND TECHNOLOGY
Thanjavur | College Code: 8219

Contents

01	Project Overview	3
02	Phase 1: Requirement Analysis and Planning	4
03	Phase 2: Backend Development and Configuration	11
04	Phase 3: UI/UX Development and Customization	20
05	Phase 4: Approvals, Security and Testing	26
06	Plan and Schedule	34
07	Strengths and Limits	36
08	Conclusion and Roadmap	37
09	Deployment and Maintenance	38
A	Appendix	40

The project at a glance

4 team members	**2** custom objects: Policy and Claim	**6** record types across both objects	**5** Flows: screen and record-triggered
1 Approval Process with 2 steps	**3** Apex classes: calculator, controller, test	**2** Lightning Web Components	**3** permission sets and 2 sharing rules

01 Project Overview

What the system is and why we built it

Insurance companies handle several product lines at the same time, and each line asks for different information. When quoting, policy issuance and claims live in registers, spreadsheets and separate tools, quotes take too long, data is entered inconsistently and claims wait in the wrong hands. The **Multi-Line Insurance Policy and Claims Management System** brings Vehicle, Property and Life insurance onto one Salesforce platform, so every customer, policy and claim sits in a single connected data model.

The application is built in a Salesforce Developer Edition org. Policy and Claim custom objects use Record Types and Field Sets to hold product-specific data. A guided Screen Flow handles auto quoting, a validation rule keeps VINs clean, and Apex calculates the premium. When a claim is created, a Record-Triggered Flow routes it to the Auto, Property or Life queue. Claims above $50,000 go through a two-step approval (Senior Adjuster, then Manager), and adjusters see their own workload in an LWC Claims Dashboard. Access is controlled with three permission sets and state-based sharing rules.

Purpose and objectives

- Centralize Vehicle, Property and Life policies and claims in one Salesforce platform.
- Standardize and speed up policy quoting and issuance with a guided Screen Flow.
- Calculate premiums automatically from policy details such as state and model year.
- Route every new claim to the right queue based on its policy type.
- Send high-value claims through a structured, multi-level approval.
- Give claims adjusters a focused dashboard of their assigned claims with amount, status and days open.
- Give each type of user only the access that their job needs, including state-based access.
- Provide a 360-degree view of customer, policy and claim information.

Technology used

Layer	Salesforce feature used
Platform	Salesforce CRM, Developer Edition org
Data model	Custom objects Policy__c and Claim__c, Record Types, Field Sets, Lookup relationships to Contact and User
Business logic	Screen Flow, Record-Triggered Flows, Validation Rule, Approval Process, Field Updates
Code	Apex class PremiumCalculator, ClaimsAdjusterController and its test class
User interface	Lightning pages, Lightning Web Components (claimsDashboardLwc, claimTileLwc), Quick Action
Security	Permission Sets and state-based Sharing Rules
Insight	Claims Dashboard, Salesforce Reports and Dashboards

02

Phase 1: Requirement Analysis and Planning

Problems, scope, stakeholders and roadmap

Problem statements

ID	Who	Is trying to ...	but ...	Because ...	Feels
PS-1	Insurance Operations Manager	standardize policy quoting and issuance	the process is manual, slow and inconsistent	legacy systems and processes are fragmented	Frustrated
PS-2	Insurance Agent	create and manage policies efficiently	different insurance lines need different information	Vehicle, Property and Life policies have diverse data	Overwhel med
PS-3	Insurance Agent	provide accurate policy quotes	quoting needs multiple steps and manual processing	there is no unified quoting platform	Uncertain
PS-4	Claims Manager	manage claims efficiently	claim review and routing are largely manual	claim processes are not centralized or automated	Stressed
PS-5	Claims Adjuster	manage assigned claims effectively	I lack a focused view of my workload	claim information is not in one place	Less productive
PS-6	Claims Manager	route claims to the right team	claims must follow policy type and severity	routing is complex and needs automation	Concerned
PS-7	Senior Adjuster	process high-value claims quickly	approval needs several levels of review	high-value claims need structured escalation	Pressured
PS-8	Insurance Operations Manager	keep policy information accurate	products need different fields and checks	policy types have varying data rules	Concerned
PS-9	Customer	get my policy and claim handled quickly	manual processing causes delays	policy and claim processes are fragmented	Dissatisfied
PS-10	Insurance Management	get a complete view of customer, policy and claim	information is spread across processes	there is no unified platform	Concerned

Empathy map: Insurance Claims Adjuster

THINKS AND FEELS Concerned about claim delays and workload	**HEARS** Customer and team concerns about claim resolution	**SEES** Many claims, policy details and approval statuses
SAYS AND DOES Reviews claims and follows up on approvals	**PAIN** Manual processing, complex routing, limited visibility	**GAIN** Claims dashboard, automation and faster approvals

Ideas considered and final choice

Idea	Approach	Key limitation
Manual registers	Policy and claim information kept by hand	Slow, error-prone and hard to maintain
Spreadsheet tracking	Policies, claims and approvals tracked in sheets	Limited automation and no real-time visibility
Standalone portal	A separate portal for policies and claims	High development and maintenance effort
Salesforce CRM (chosen)	One platform for policies, claims, automation and reporting	Needs careful configuration, which this project provides

Decision

Salesforce CRM was chosen for automation (quoting, premium calculation, routing and approvals), scalability (Vehicle, Property, Life and future lines), real-time visibility, one unified platform and less manual processing.

Functional scope

- **Data model:** two custom objects, Policy and Claim, linked to Contact (customer) and User (adjuster).
- **Flexible data:** Record Types for Auto, Property and Life policies and Accident, Property and Life claims, with Field Sets for each.
- **Quoting:** an Auto Quoting Screen Flow that creates a draft policy, with a 17-character VIN check.
- **Apex:** a PremiumCalculator invocable class and a ClaimsAdjusterController for the dashboard.
- **Automation:** claim routing to queues, a $50,000 approval trigger and a state-update flow.
- **UI:** an LWC Claims Dashboard with a reusable claim tile and an Approve/Reject quick action.
- **Security:** three permission sets and sharing rules for the CA and TX states.

Stakeholders

Stakeholder	What they do in the system
Insurance Agent	Quotes policies through the Screen Flow and manages customer and policy records.
Claims Adjuster	Works on assigned claims through the Claims Dashboard and reviews claim details.
Senior Adjuster	First approver for high-value claims.
Claims Manager	Second approver, reporting and oversight of claim handling.
System Administrator	Maintains objects, flows, security and access.
Customer	Requests a policy, raises claims and expects quick, accurate handling.

Customer journey

#	Stage	What happens in Salesforce
1	Insurance inquiry	The customer asks about cover and the agent opens the quoting flow.
2	Policy selection	Vehicle, Property or Life is chosen through the matching Record Type.
3	Quote request	The Auto Quoting Screen Flow starts a guided quote.
4	Policy information capture	Policy holder, start date, state, VIN and model year are entered.
5	Premium calculation	PremiumCalculator Apex returns the premium for the draft policy.
6	Policy issuance	The validated policy is stored in Policy__c.
7	Claim initiation	A Claim__c record is created and linked to the policy.
8	Claim routing	A Record-Triggered Flow assigns the claim to the Auto, Property or Life queue.
9	Review and approval	Claims above $50,000 go to the Senior Adjuster, then the Manager.
10	Claim resolution	The approval decision updates the Approval Status field.
11	Customer confirmation	The customer is informed of the outcome by the claims team.
12	Reporting and visibility	The Claims Dashboard and reports show the live position.

Solution requirements

Functional requirements

No.	Requirement
FR-01	Create and manage Insurance Policy records for Vehicle, Property and Life insurance.
FR-02	Support separate Record Types for Vehicle, Property and Life policies.
FR-03	Capture product-specific data such as VIN for Vehicle and Square Footage / Year Built for Property.
FR-04	Provide a guided Auto Quoting Screen Flow capturing policy holder, start date, state, VIN and model year.
FR-05	Create a draft policy during the quoting process.
FR-06	Calculate policy premiums with Apex-based logic.
FR-07	Validate that a Vehicle VIN contains exactly 17 characters.
FR-08	Create and manage Claim records associated with policies.
FR-09	Support Claim Record Types for Accident, Property and Life claims.
FR-10	Route claims automatically to the right queue based on the related policy type.
FR-11	Identify high-value claims using the configured amount threshold.
FR-12	Send qualifying claims through a multi-level approval (Senior Adjuster, then Claims Manager).
FR-13	Provide Approve/Reject with approver comments.
FR-14	Provide Claims Adjusters an LWC Claims Dashboard of assigned claims.
FR-15	Show Claim Number, Policy Type, Policy Holder, Claim Amount, Status and Days Open.
FR-16	Allow filtering of claims by All, Auto, Property and Life.
FR-17	Update related Policy and Claim data through Record-Triggered Flows.
FR-18	Support state-based access and sharing.
FR-19	Provide role-specific Permission Sets for Agents, Adjusters and Managers.
FR-20	Support reporting and visibility of customer, policy and claim information.

Non-functional requirements

ID	Quality	Description
NFR-01	Usability	Simple guided Screen Flows that cut unnecessary clicks for agents.
NFR-02	Performance	Quoting, premium calculation, routing and dashboard actions run efficiently.
NFR-03	Scalability	Supports Vehicle, Property and Life now and more products later.
NFR-04	Data integrity	Policy and claim data is validated so records stay accurate and consistent.
NFR-05	Security	Access follows responsibility, with state-based sharing and Permission Sets.
NFR-06	Maintainability	Modular Flows, Apex, LWC, Record Types and Field Sets that can be changed independently.
NFR-07	Reliability	Routing, premium calculation and approvals run consistently by the configured rules.
NFR-08	Extensibility	Product-specific fields fit the same structure without a new model per line.
NFR-09	Visibility	Timely access to customer, policy and claim data through records and the dashboard.
NFR-10	User adoption	An intuitive experience that reduces manual effort for agents and claims users.

Solution architecture and data flow

The diagram shows how users reach the system and how each layer builds on the one below it. Supporting flows run alongside the main flow: **quoting** (inputs, validation, draft policy, premium), **claim routing** (claim created, policy type read, queue assigned) and **approval** (amount over $50,000, Senior Adjuster, Claims Manager, status update).

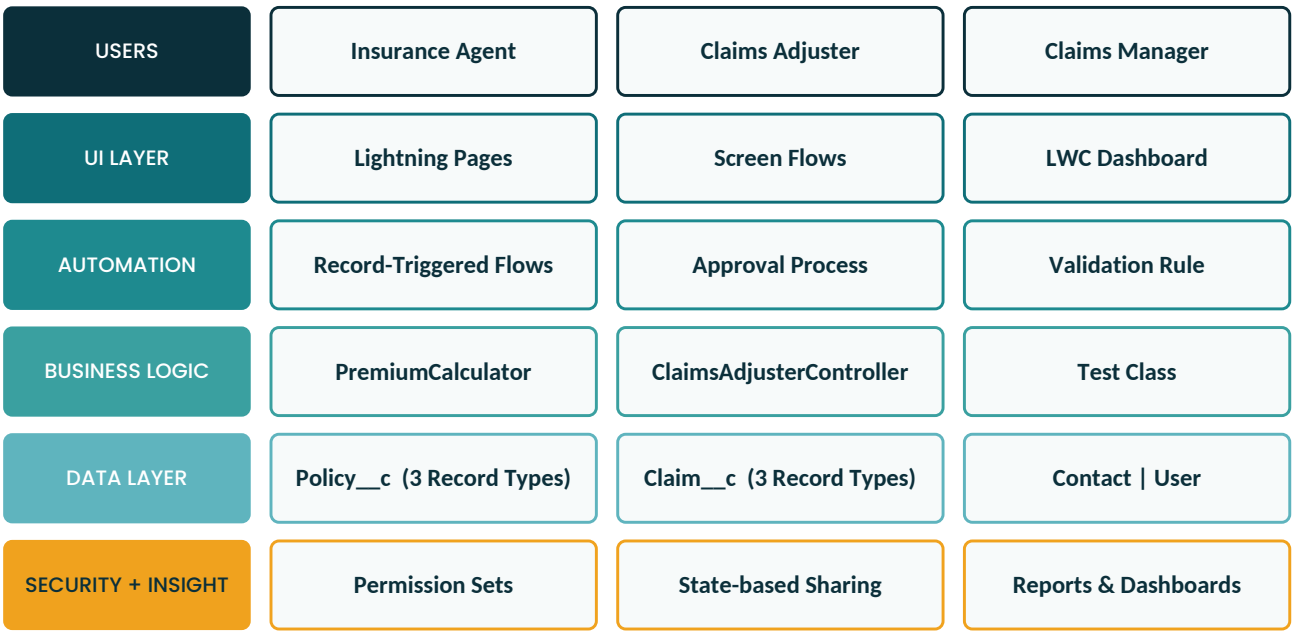


Figure A. Layered solution architecture of the Insurance Policy and Claims system.

Main data flow

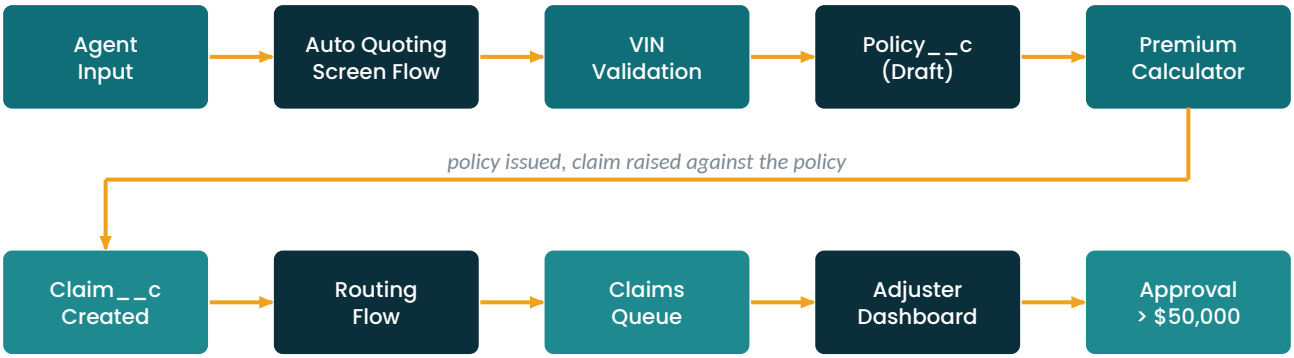


Figure B. Main data flow from agent input to approval and dashboard.

Data model

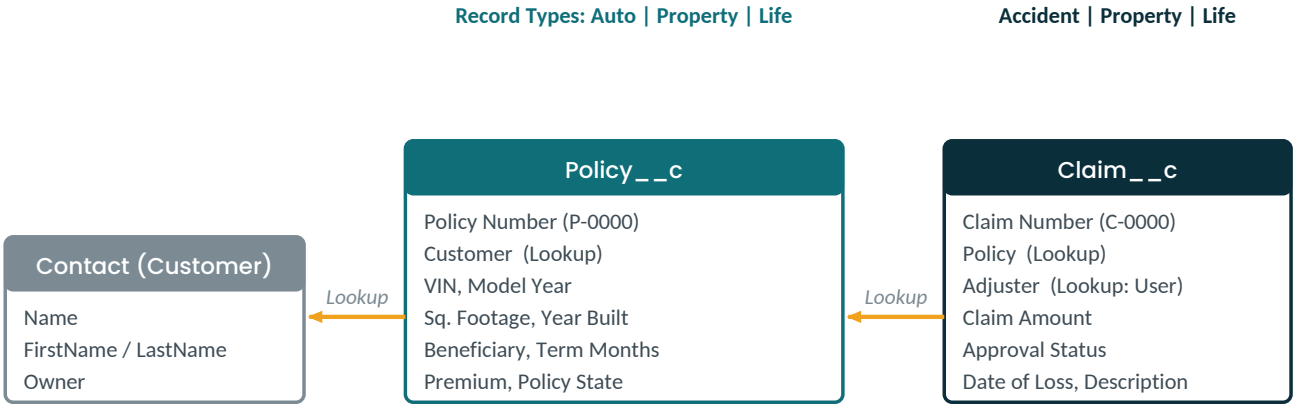


Figure C. Data model with Policy__c, Claim__c and the Contact and User lookups.

Data flow steps

Step	What happens
1. Input	The agent provides customer and policy information.
2. Capture	The AutoQuoting Screen Flow captures policy holder, start date, state, VIN and model year.
3. Validation	Salesforce validates the data, including the 17-character VIN rule for Auto policies.
4. Policy creation	Valid data is stored in Policy__c under the Auto, Property or Life Record Type.
5. Premium	PremiumCalculator Apex calculates the premium from the policy data.
6. Claim creation	A Claim__c record is created and linked to the policy.
7. Routing	A Record-Triggered Flow reads the policy type and routes the claim to the right queue.
8. Review	The adjuster opens assigned claims in the Claims Dashboard.
9. Approval	Claims above the threshold go to the Senior Adjuster, then the Claims Manager.
10. Decision	The approver chooses Approve or Reject with comments and the status updates.
11. Reporting	The dashboard shows claim amount, policy type, holder, status and days open.

Problem and solution fit

Problem	Our solution	Benefit
Manual, inconsistent quoting	Auto Quoting Screen Flow	Guided, standard quotes
Different data per product	Record Types and Field Sets	One model, many products
Wrong VINs entered	VIN_Must_Be_17_Characters validation rule	Clean vehicle data
Manual premium work	PremiumCalculator Apex	Consistent premiums
Slow claim assignment	Claim routing Record-Triggered Flow	Right queue instantly
High-value claims unreviewed	Two-step approval process and flows	Controlled escalation
No adjuster workload view	claimsDashboardLwc and claimTileLwc	Focused daily view
Different access needs	Permission Sets and state sharing rules	Secure, role-based access

Execution roadmap

Step	Work item	Phase
1	Create the Developer org, the Policy and Claim objects, Record Types and fields	Phase 2
2	Build the AutoQuoting Screen Flow and the VIN validation rule	Phase 2
3	Write PremiumCalculator and connect it to the quoting flow	Phase 2
4	Build the claim routing Record-Triggered Flow	Phase 2
5	Write ClaimsAdjusterController and the two LWC components	Phase 3
6	Build the High Value Claim Approval process, submit flow and approver flow	Phase 4
7	Add the state-update flow, sharing rules and permission sets	Phase 4
8	Write the Apex test class, run the tests and record results	Phase 4

03 Phase 2: Backend Development and Configuration

Org setup, data model, quoting, premium and routing

This phase built the working engine of the system. Declarative tools (Record Types, Field Sets, Screen Flow, validation rule and Record-Triggered Flow) handle the routine rules, and Apex handles the premium calculation.

Milestone 1: Salesforce Developer org

- Open the Salesforce developer sign-up page and fill in first name, last name, email, role (Developer), company (college name), country (India), postal code and a unique username in the form *name@organization.com*.
- Open the verification email (it can take 5 to 10 minutes), click **Verify Account**, set a password and answer the security question.
- You are then taken to Salesforce Setup. All objects, automation and code in this project were built in this org.

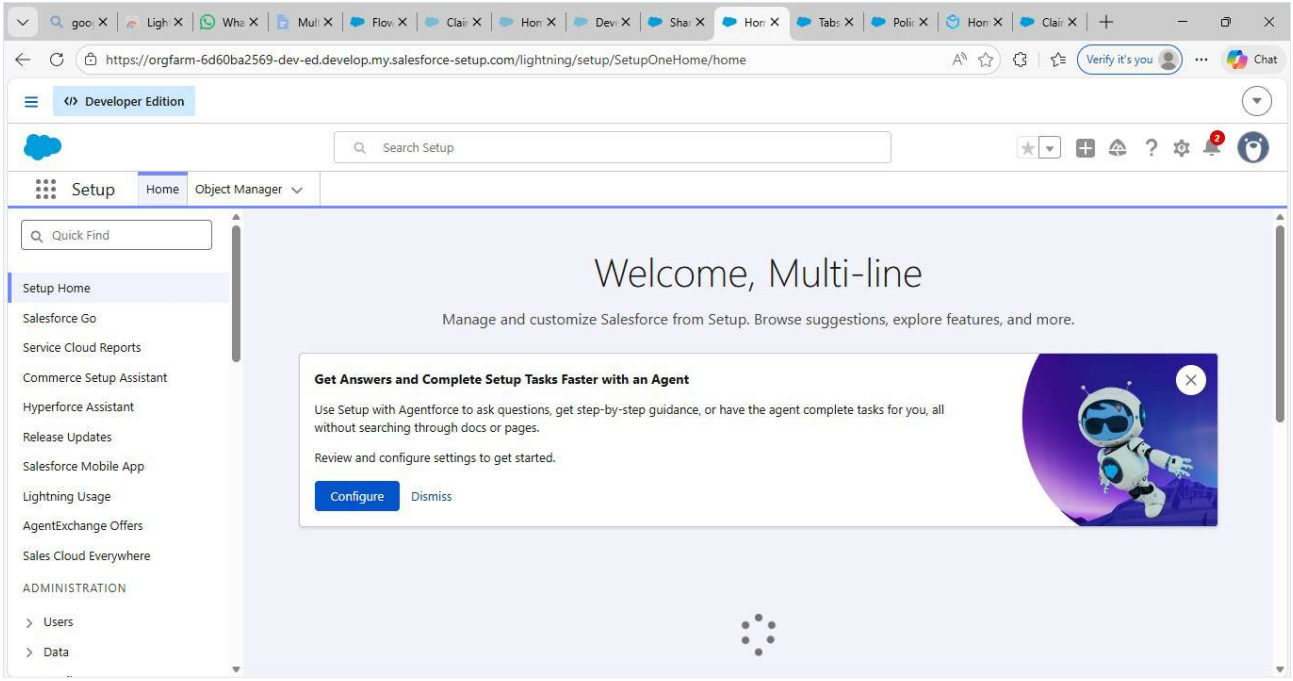


Figure 1. *Salesforce Setup home of the Developer Edition org used for this project.*

Milestone 2: Custom objects

Two custom objects were created from **Setup > Object Manager > Create > Custom Object**. For both, *Allow Reports*, *Allow Activities* and *Track Field History* were enabled, the object was set to **Deployed** and *Allow Search* was checked.

Setting	Policy	Claim
Label / Plural label	Policy / Policies	Claim / Claim
Record name	Policy Number	Claim Number
Data type	Auto Number	Auto Number

Setting	Policy	Claim
Display format	P-{0000}	C-{0000}
Starting number	1	1

A custom object tab was also created for Claim (Setup > Tabs > Custom Object Tabs > New) so that users can open claim records quickly.

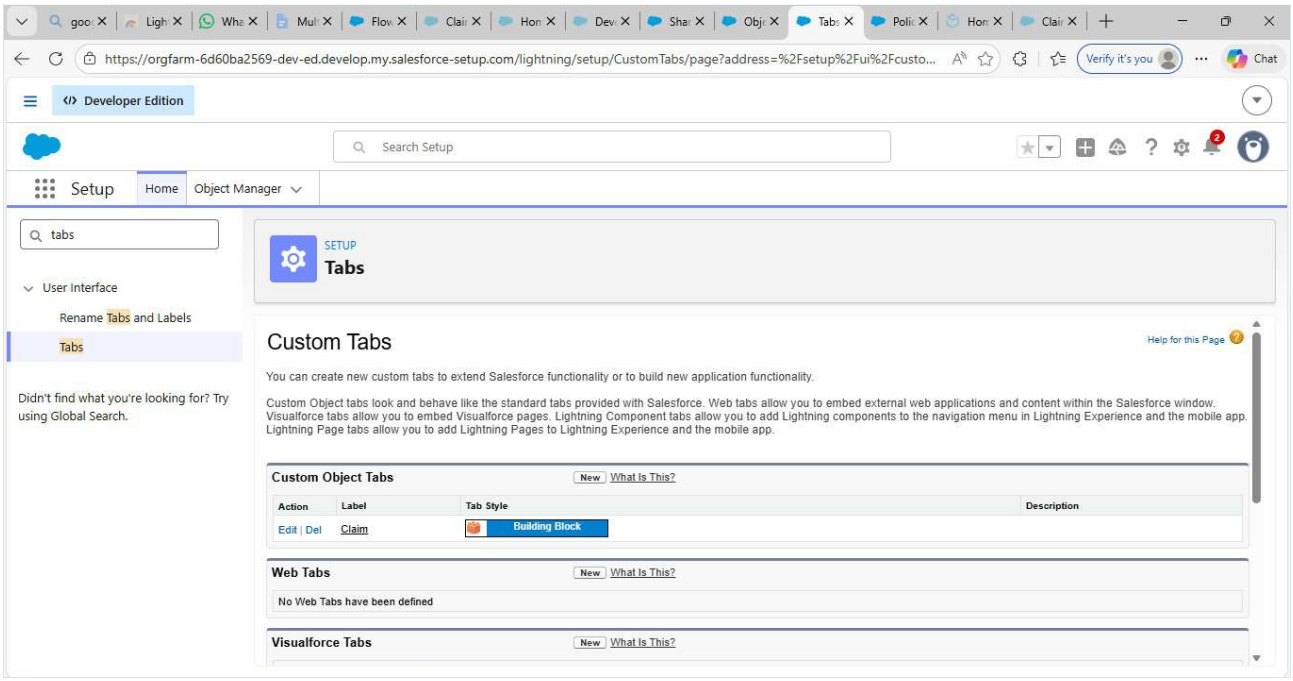


Figure 2. Custom Object Tabs in Setup showing the Claim tab.

Milestone 3: Record Types and Field Sets

Record Types let one Policy object serve three insurance lines. On Policy__c, the Record Types **Auto**, **Property** and **Life** were created from Object Manager > Record Types > New (based on Master, active, available to all profiles, default page layout). The same was done on Claim__c for **Accident**, **Property** and **Life**.

Field Sets group the fields that belong to each product. They were created from Object Manager > Policy > Field Sets > New, with the field set *used in the Policy object*:

Field Set	Fields included
Vehicle	VIN, Model Year
Property	Square Footage, Year Built
Life	Beneficiary Name, Policy Term Months

Milestone 4: Fields and relationships

Policy object fields

#	Field name	Data type
1	Beneficiary Name	Text (50)
2	Customer	Lookup (Contact)
3	Model Year	Text (5)
4	Policy Start Date	Date
5	Policy Term Months	Number (18, 0)
6	Premium	Currency (16, 2)
7	Square Footage	Number (18, 0)
8	VIN	Text (20)
9	Year Built	Text (5)
10	Policy State	Picklist (used by the quoting flow, premium logic and sharing)

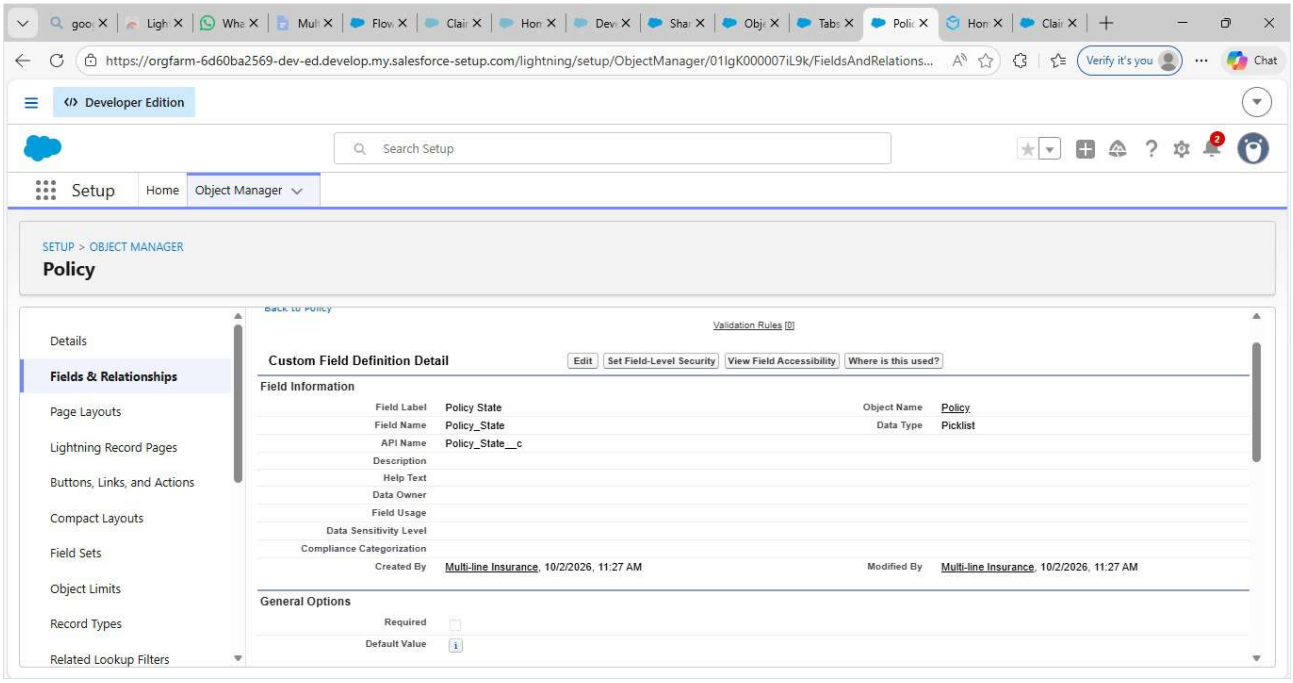


Figure 3. *Policy object: Policy State picklist field definition (API name Policy_State__c).*

Claim object fields

#	Field name	Data type
1	Adjuster	Lookup (User)
2	Approval Status	Picklist (New, Submitted for Approval, Approved, Rejected)
3	Claim Amount	Currency (16, 2)
4	Date of Loss	Date/Time
5	Description	Long Text Area
6	Policy	Lookup (Policy)
7	Policy Account Holder State	Text (filled by the state-update flow in Phase 4)

Relationship	Type	Purpose
Policy to Contact (Customer)	Lookup	Each policy belongs to a customer
Claim to Policy	Lookup	Each claim is raised against a policy
Claim to User (Adjuster)	Lookup	Each claim is handled by an adjuster

Milestone 5: Auto Quoting Screen Flow

The flow **AutoQuotingFlow** (Setup > Flows > New Flow > Screen Flow) walks an agent through a quote in four elements.

Element	Configuration
Get Records: Get Vehicle RT ID	Object: Record Type. Conditions (all met): sObject Type Name equals Policy__c and Record Type Name equals Auto.
Screen: New Auto Policy Quote - Basic Information	Lookup component Policy Holder (Customer__c on Policy__c), Date component Policy Start Date, Picklist component Policy State (picklist choice set on Policy > Policy State).
Screen: Vehicle-Specific Details	Text component VIN and text component Model year.
Create Records: Create Draft Policy	Policy object with Customer, Model Year, Policy Start Date, Policy State, VIN and RecordTypeId set from the screens and from Get Vehicle RT ID. The new Id is stored in the variable `varPolicyId`.

Flow saved and active
The flow was saved as **AutoQuotingFlow** and activated. After Milestone 7 it also calculates and stores the premium.

Milestone 6: Validation rule on Policy

To stop incorrect vehicle data, the Policy object has an active validation rule **VIN_Must_Be_17_Characters**. It applies to the Auto Record Type, shows its message on the VIN field and blocks the save when the VIN length is not 17.

Error condition formula

```
AND( RecordType.DeveloperName = "Auto", LEN( VIN__c ) <> 17 )
```

Error message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.

Milestone 7: Premium calculation with Apex

The Apex class **PremiumCalculator** was created in the Developer Console (File > New > Apex Class). It has an `@InvocableMethod` so that a Flow can call it. The base premium is 1000. It is raised by 15% for California (CA) and 5% for Texas (TX), and 100 is added when the model year is before 2018.

PremiumCalculator.cls

```
public class PremiumCalculator {
  public class PremiumInputWrapper {
    @InvocableVariable(required=true)
    public Id policyId;
  }
  @InvocableMethod(label='Calculate Policy Premium'
    description='Calculates premium based on policy fields (VIN, Model Year, State).')
  public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
    List<Decimal> premiumResults = new List<Decimal>();
    Set<Id> policyIds = new Set<Id>();
    for (PremiumInputWrapper input : inputs) {
      if (input.policyId != null) {
        policyIds.add(input.policyId);
      }
    }
    List<Policy__c> policiesToRate = [
      SELECT Id, VIN__c, Model_Year__c, Policy_State__c
      FROM Policy__c
      WHERE Id IN :policyIds
    ];
    for (Policy__c policy : policiesToRate) {
      Decimal basePremium = 1000.00;
      if (policy.Policy_State__c == 'CA') {
        basePremium *= 1.15;
      } else if (policy.Policy_State__c == 'TX') {
        basePremium *= 1.05;
      }
      if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018) {
        basePremium += 100;
      }
      premiumResults.add(basePremium);
    }
    return premiumResults;
  }
}
```

Connecting the calculator to the quoting flow

In Flow Builder, AutoQuotingFlow was opened and extended after the Create Records element:

- **Action: Calculate Premium Action** calls PremiumCalculator with `varPolicyId` as the policy Id. The output is stored in a new currency variable `varCalculatedPremium` (2 decimal places).
- **Update Records: Update Policy With Premium** finds the Policy where Record Id equals `varPolicyId` and sets Premium to `varCalculatedPremium`.
- The flow was saved as a new version and activated.

Milestone 8: Claim routing flow

A Record-Triggered Flow on the Claim object (After Save, run when a record is **created**, optimized for Actions and Related Records) sends each new claim to the right queue.

Element	Configuration
Get Records: Get_Policy_RT	Policy where Id equals the triggering Claim's Policy. First record only; the Record Type value is stored in `varPolicyRTName` (Text).
Decision: Route by Policy Type	Outcome **Auto Claim Route** when varPolicyRTName equals Auto. Outcome **Property Claim Route** when it equals Property. Default outcome **Life Claim Route**.
Update Records (one per path)	**Assign to Auto Queue**, **Assign to Property Queue**, **Assign to Life Queue**. Each updates the Claim whose Id equals the triggering record and sets OwnerId to the 18-character Id of its queue.

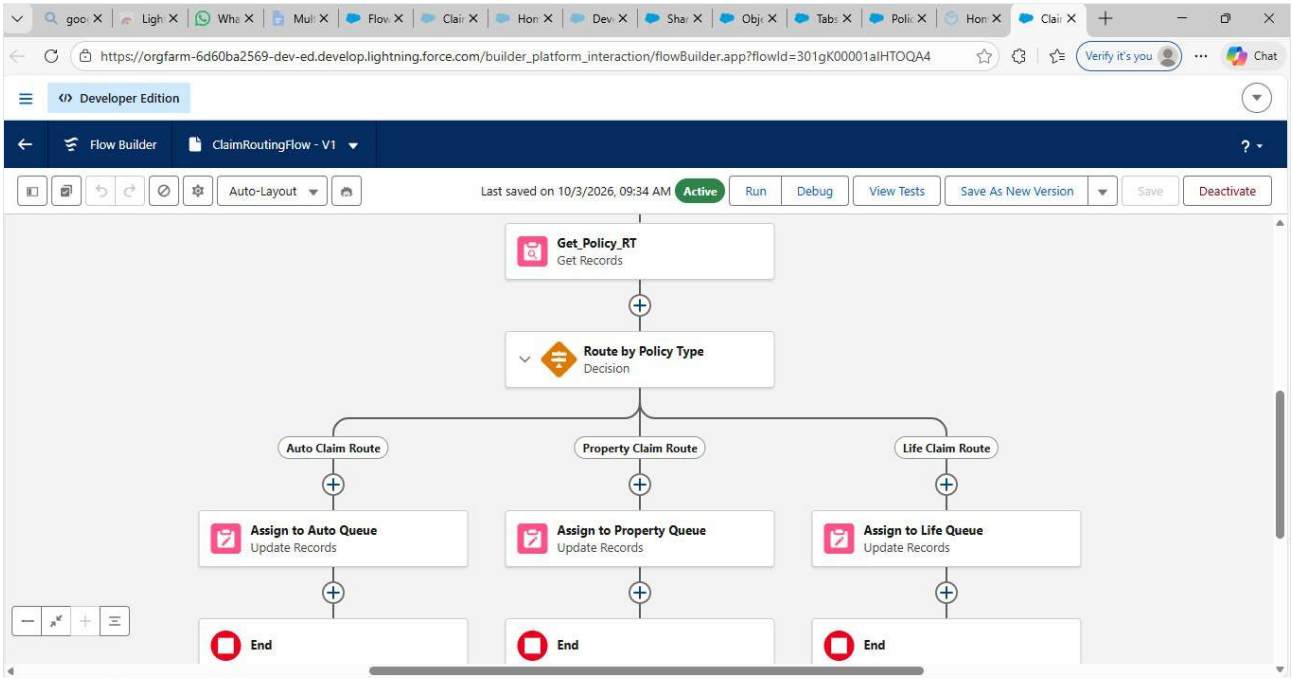


Figure 4. Claim routing flow: Get Records, Decision and three queue assignments.

04 Phase 3: UI/UX Development and Customization

The Claims Adjuster controller and Lightning Web Components

Adjusters work from one screen. A controller class reads the claims owned by the logged-in user, and two Lightning Web Components turn that data into a filterable dashboard of claim tiles.

Milestone 9: ClaimsAdjusterController

The class **ClaimsAdjusterController** runs a SOQL query for all claims owned by the current user, joining the related Policy and Customer data, and returns a list of `ClaimWrapper` objects with the key metrics for each claim. The method is `@AuraEnabled(cacheable=true)` so the LWC can call it with `@wire`.

ClaimsAdjusterController.cls

```
public class ClaimsAdjusterController {
  public class ClaimWrapper {
    @AuraEnabled public Id claimId;
    @AuraEnabled public String claimNumber;
    @AuraEnabled public Decimal claimAmount;
    @AuraEnabled public String status;
    @AuraEnabled public String policyType;
    @AuraEnabled public String policyHolderName;
    @AuraEnabled public Integer daysOpen;

    public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
      this.claimId = c.Id;
      this.claimNumber = c.Name;
      this.claimAmount = c.Claim_Amount__c;
      this.status = c.Approval_Status__c;
      Policy__c policyRecord = (Policy__c)p;
      if (policyRecord != null && policyRecord.RecordType != null) {
        this.policyType = policyRecord.RecordType.Name;
      } else {
        this.policyType = 'Unknown';
      }
      this.policyHolderName = (customerRec != null) ? customerRec.Name : '';
      this.daysOpen = (c.Date_of_Loss__c != null)
        ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
        : 0;
    }
  }

  @AuraEnabled(cacheable=true)
  public static List<ClaimWrapper> getAssignedClaims() {
    List<ClaimWrapper> results = new List<ClaimWrapper>();
    List<Claim__c> claims = [
      SELECT Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c,
             Policy__r.Name,
             Policy__r.RecordType.DeveloperName,
             Policy__r.Customer__r.FirstName,
             Policy__r.Customer__r.LastName
      FROM Claim__c
      WHERE OwnerId = :UserInfo.getUserId()
      ORDER BY Date_of_Loss__c DESC
    ];
    for (Claim__c claim : claims) {
      Policy__c policy = claim.Policy__r;
      Contact customer = (policy != null && policy.Customer__r != null)
        ? (Contact)policy.Customer__r
        : new Contact();
      results.add(new ClaimWrapper(claim, policy, customer));
    }
```

ClaimsAdjusterController.cls (continued)

```
    return results;
  }
}
```

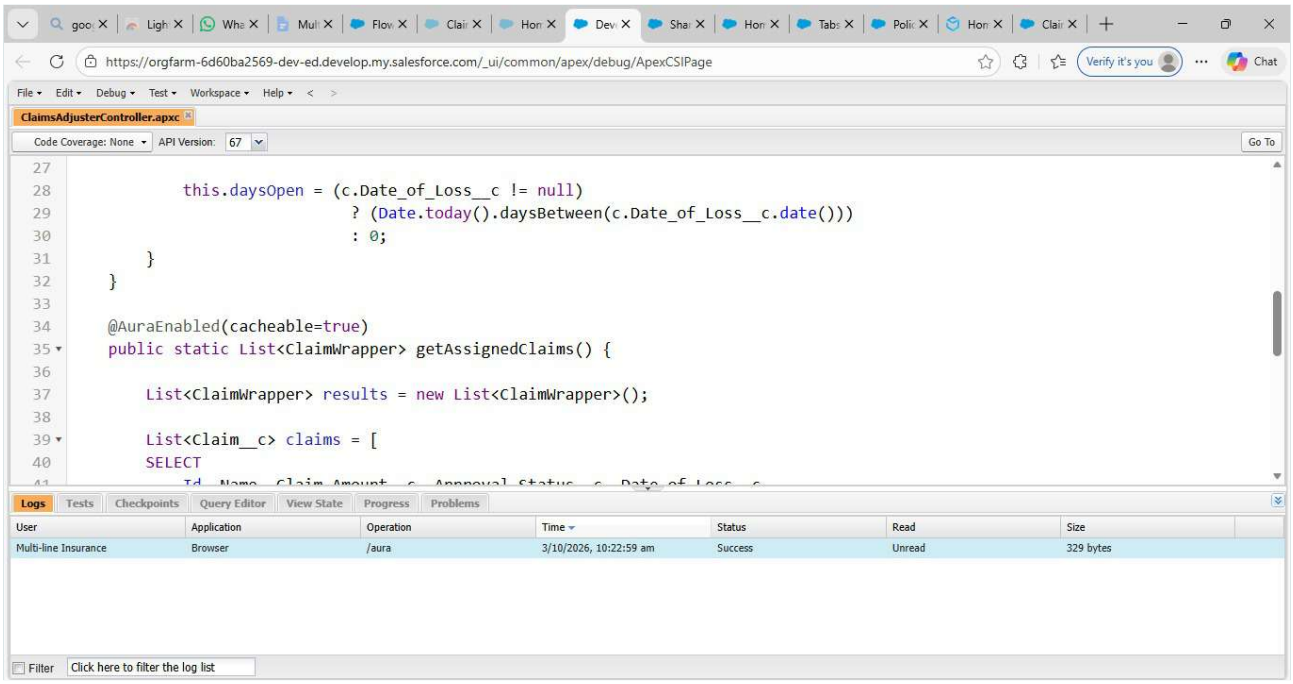


Figure 5. ClaimsAdjusterController saved in the Developer Console.

Milestone 10: claimsDashboardLwc

The component **claimsDashboardLwc** was created in the Lightning Studio browser extension (Create New > Lightning Web Component, *isExposed* checked, targets Lightning app page, Home page and Record page) and deployed. It loads the claims with `@wire`, keeps two lists (all claims and visible claims) and filters on the client when the user changes the **Filter by Policy Type** combobox (All, Auto, Property, Life).

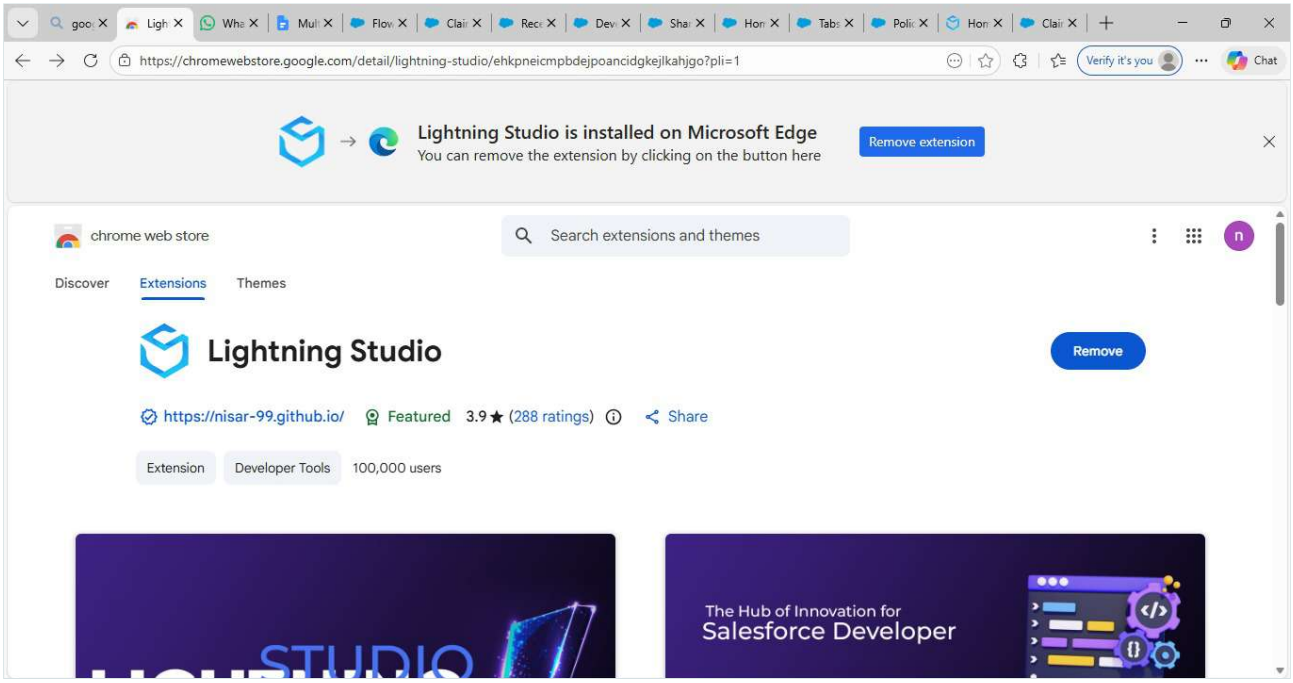


Figure 6. Lightning Studio extension installed in the browser, used to create and deploy the LWC components.

claimsDashboardLwc.html

```
<template>
  <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">
    <div class="slds-m-around_medium">
      <lightning-combobox
        name="policyFilter"
        label="Filter by Policy Type"
        value="All"
        options={policyTypeOptions}
        onchange={handleFilterChange}
        class="slds-m-bottom_medium">
      </lightning-combobox>
      <template if:true={visibleClaims}>
        <div class="slds-grid slds-wrap slds-grid_pull-padded">
          <template for:each={visibleClaims} for:item="claim">
            <div key={claim.claimId}
              class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3 slds-p-hor
              <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>
            </div>
          </template>
        </div>
        <p class="slds-m-top_medium slds-text-heading_small">
          Total Claims Assigned: {visibleClaims.length}
        </p>
      </template>
      <template if:false={visibleClaims}>
        <p>No claims currently assigned to you.</p>
      </template>
    </div>
  </lightning-card>
</template>
```

claimsDashboardLwc.js

```
import { LightningElement, wire, track } from 'lwc';
import getAssignedClaims from '@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
  @track allClaims = [];
  @track visibleClaims = [];
  error;
```

claimsDashboardLwc.js (continued)

```
  @wire(getAssignedClaims)
  wiredClaims({ error, data }) {
    if (data) {
      this.allClaims = data;
      this.visibleClaims = data;
      this.error = undefined;
    } else if (error) {
      this.error = error;
      this.allClaims = undefined;
      this.visibleClaims = undefined;
      console.error('Error retrieving claims:', JSON.stringify(error));
    }
  }

  handleFilterChange(event) {
    const selectedPolicyType = event.target.value;
    if (selectedPolicyType === 'All') {
      this.visibleClaims = this.allClaims;
    } else {
      this.visibleClaims = this.allClaims.filter(
        claim => claim.policyType === selectedPolicyType
      );
    }
  }

  get policyTypeOptions() {
    return [
      { label: 'All Policy Types', value: 'All' },
      { label: 'Auto', value: 'Auto' },
      { label: 'Property', value: 'Property' },
      { label: 'Life', value: 'Life' }
    ];
  }
}
```

claimsDashboardLwc.js-meta.xml

```
<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>63.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>
```

Milestone 11: claimTileLwc

The reusable child component **claimTileLwc** shows one claim: claim number, policy type with an icon (truck for Auto, home for Property, connected apps for Life), policy holder, claim amount as currency, status and days open. The parent passes the data in with an `@api claimData` property.

claimTileLwc.html

```
<template>
  <lightning-layout-item size="12" padding="around-small">
    <div class="tile slds-box slds-p-around_medium slds-theme_default">
      <header class="slds-media slds-media_center slds-m-bottom_small">
        <div class="slds-media__figure">
          <lightning-icon icon-name={policyIcon}
            alternative-text={claimData.policyType} size="small"></lightning-icon>
        </div>
        <div class="slds-media__body slds-m-left_small">
          <h2 class="slds-text-heading_medium slds-m-bottom_none">
            Claim: {claimData.claimNumber}
          </h2>
          <p class="slds-text-body_small">Policy Type: <strong>{claimData.policyType}</strong></p>
        </div>
      </header>
      <div class="slds-border_top slds-p-top_small">
        <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p>
        <lightning-formatted-number value={claimData.claimAmount}
          format-style="currency" currency-code="USD" label="Claim Amount">
        </lightning-formatted-number>
        <p>Status: {claimData.status}</p>
        <p class="slds-text-body_regular">
          Days Open: <strong class="slds-text-color_error">{claimData.daysOpen}</strong>
        </p>
      </div>
    </div>
  </lightning-layout-item>
</template>
```

claimTileLwc.js

```
import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
  @api claimData;

  get policyIcon() {
    if (this.claimData && this.claimData.policyType) {
      const type = this.claimData.policyType.toLowerCase();
      if (type === 'auto') return 'utility:truck';
      if (type === 'property') return 'utility:home';
      if (type === 'life') return 'utility:connected_apps';
    }
    return 'utility:help';
  }
}
```

Result

Adjusters see only the claims they own, with claim number, policy type, holder, amount, status and days open, and can narrow the view to Auto, Property or Life with one click.

05 Phase 4: Approvals, Security and Testing

High-value claim approval, access control and the test plan

Milestone 12: Approval Status field and approval process

A picklist field **Approval Status** (New, Submitted for Approval, Approved, Rejected) tracks where each claim stands. The approval process **High Value Claim Approval** was created on the Claim object from Setup > Approval Processes > Create New Approval Process > Use Standard Setup Wizard.

Setting	Value
Entry criteria	Claim Amount greater than 50000
Record editability	Administrators ONLY can edit records during approval
Approval page layout fields	Claim Number, Owner, Claim Amount, Adjuster, Policy
Security	Approvers can access the page only from within Salesforce
Initial submitters	Claim Owner

Approval steps

Step	Configuration
Step1 Senior Adjuster	All records enter the step. Approver is assigned automatically to the Senior Adjuster user. With several approvers, the decision follows the FIRST response.
Step2 Manager Review	All records enter the step. Approver is the Department (Claims) Manager user, FIRST response decides. On reject: perform all rejection actions for this step AND all final rejection actions.

Field updates

Stage	Field update	New value of Approval Status
Initial submission	Update Status Submitted	Submitted for Approval
Final approval	Update Status Approved	Approved
Final rejection	Update Status Rejected	Rejected

Milestone 13: Submit-to-approval flow

A Record-Triggered Flow (After Save) on Claim starts the approval automatically. It runs when a record is created or updated, with the entry conditions **Claim Amount > 50000** and **Approval Status = New**, every time the record is updated and meets the conditions, optimized for Actions and Related Records. An Action element **Submit Claim to Approval** passes the triggering Claim's Record Id and the approval process name *High Value Claim Approval*. The flow is saved as **Submission Automation Flow** and activated.

Milestone 14: Approver screen flow and quick action

Approvers can decide from the claim record itself. The Screen Flow **Claim Approver Screen Flow** takes the record Id in a text variable `recordId` (available for input) and works like this:

- **Get Records: Get Claim Details** finds the Claim whose Id equals recordId and stores all fields.
- **Screen: Claim Review Screen** shows the claim name, amount and policy type as display text, a radio group **Approval Decision** (choices Approve and Reject) and a long text field **Approver Comments**.
- **Decision: Check Decision** has the outcome *Decision Approved* when the radio value equals Approve; the default outcome is *Decision Rejected*.
- Each path runs a **Submit for Approval** action (*Approve Claim* or *Reject Claim*) with recordId and the approver comments.

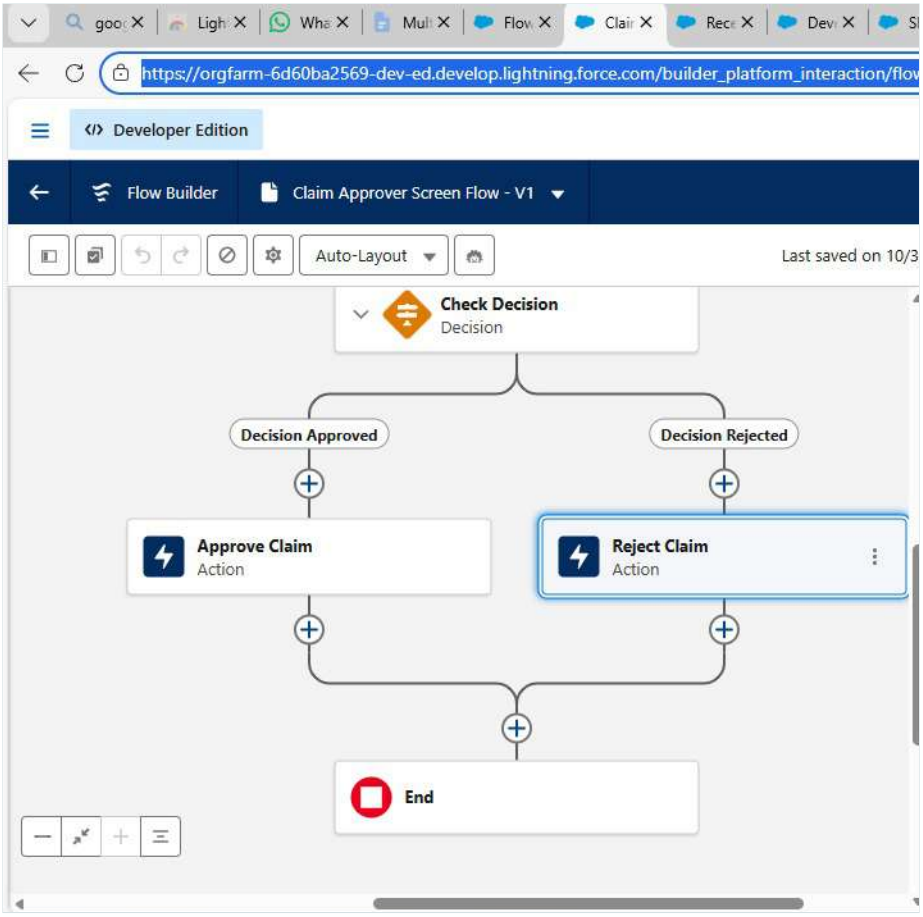


Figure 7. Claim Approver Screen Flow: review screen, decision and the approve and reject actions.

A Quick Action was then created (Object Manager > Claim > Buttons, Links and Actions > New Action): type **Flow**, flow *Claim Approver Screen Flow*, label **Approve/Reject Claim**. It was dragged into *Salesforce Mobile and Lightning Experience Actions* on the Claim page layout.

Milestone 15: State update flow and sharing rules

Adjusters should only see claims in their territory. First, a Record-Triggered Flow **Claim Policy Holder State Update** (created or updated, no entry conditions, optimized for Updates) copies the policy state onto the claim: *Get Records get_policy_records* reads the policy state, a *Loop Claim_Filed_update* goes through the policies, and *Update Records Claim_Filed_state_update* stores the loop item's Policy State in the claim field **Policy Account Holder State**.

Then two criteria-based sharing rules were added in Setup > Sharing Settings > Claim Sharing Rules:

Rule label	Criteria	Access
Share Records based on Policy State CA	Policy Account Holder State equals CA	Read/Write
Share Records based on Policy State	Policy Account Holder State equals TX	Read/Write

Milestone 16: Permission sets

Three permission sets (Setup > Permission Sets > New) separate what agents, adjusters and managers can do.

Permission set	Object access	Other permissions
Insurance Agent Access	Policy: Create, Read. Contact: Read, Edit. Claim: Read only.	Flow User enabled; PremiumCalculator Apex class enabled.
Claims Adjuster Access	Policy: Read only. Claim: Read, Edit.	ClaimsAdjusterController Apex class enabled.
Claims Manager Access	Policy: Read only. Contact: Read, Edit. Claim: Read, Edit, Delete.	Run Reports and Manage Approvals enabled.

Milestone 17: Apex test class

The class **ClaimsAdjusterControllerTest** tests the dashboard controller. A `@testSetup` method creates a test user, a Contact, an Auto Policy and an Accident Claim, all owned by that user. The test then runs as that user, calls `getAssignedClaims()` and asserts the claim Id, number, status, amount, policy holder name and policy type. The target is 95% or more code coverage.

ClaimsAdjusterControllerTest.cls

```
@isTest
private class ClaimsAdjusterControllerTest {
  @testSetup
  static void setupTestData() {
    Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1];
    User u = new User(
      FirstName = 'Test', LastName = 'User', Alias = 'tuser',
      Username = 'testuser1234343@example.com.test',
      Email = 'testuser123@example.com',
      TimeZoneSidKey = 'America/Los_Angeles', LocaleSidKey = 'en_US',
      EmailEncodingKey = 'UTF-8', LanguageLocaleKey = 'en_US',
      ProfileId = p.Id
    );
    insert u;

    Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfosByDeveloperName()
      .get('Auto').getRecordTypeId();
    Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfosByDeveloperName()
      .get('Accident').getRecordTypeId();

    Contact con = new Contact(FirstName = 'John', LastName = 'Doe', OwnerId = u.Id);
    insert con;
```

ClaimsAdjusterControllerTest.cls (continued)

```
    Policy__c policy = new Policy__c(
      Customer__c = con.Id, RecordTypeId = policyRT,
      OwnerId = u.Id, VIN__c = '1A903284375893532'
    );
    insert policy;

    Claim__c claim = new Claim__c(
      Policy__c = policy.Id, RecordTypeId = claimRT,
      Claim_Amount__c = 5000,
      Date_of_Loss__c = Date.today().addDays(-10),
      OwnerId = u.Id
    );
    insert claim;
  }

  @isTest
  static void testGetAssignedClaims() {
    User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
    System.runAs(u) {
      Test.startTest();
      List<ClaimsAdjusterController.ClaimWrapper> results =
        ClaimsAdjusterController.getAssignedClaims();
      Test.stopTest();

      ClaimsAdjusterController.ClaimWrapper wrap = results[0];
      System.assertNotEquals(null, wrap.claimId);
      System.assertNotEquals(null, wrap.claimNumber);
      System.assertEquals('New', wrap.status);
      System.assertEquals(5000, wrap.claimAmount);
      System.assert(wrap.policyHolderName.contains('John'));
      System.assertNotEquals('Unknown', wrap.policyType);
    }
  }
}
```

To run it, open the class in the Developer Console, click **Run Test** (top right) and read the coverage in the Tests tab.

Functional and performance testing

The table lists the checks for the main features. The **Result** column is ticked after each test is run in the org, with a screenshot kept as evidence.

ID	Feature	Test steps	Expected result	Result
TC-01	Auto quoting flow	Run AutoQuotingFlow with a customer, start date, state, VIN and model year.	A draft Auto policy is created.	[ ] Pass
TC-02	VIN validation	Save an Auto policy with a 10-character VIN.	Save is blocked with the VIN error message.	[ ] Pass
TC-03	Premium calculation	Quote a policy for state CA with model year 2015.	Premium = 1000 x 1.15 + 100 = 1250.	[ ] Pass
TC-04	Claim routing (Auto)	Create a claim on an Auto policy.	Claim owner becomes the Auto queue.	[ ] Pass
TC-05	Claim routing (Property / Life)	Create claims on a Property and a Life policy.	Claims go to the Property and Life queues.	[ ] Pass
TC-06	High-value approval	Create a claim of 60000 with status New.	Claim is submitted; status becomes Submitted for Approval.	[ ] Pass
TC-07	Approve / Reject	Use the Approve/Reject Claim action on a submitted claim.	Status becomes Approved or Rejected with comments.	[ ] Pass
TC-08	Claims dashboard	Open the dashboard as an adjuster and use the filter.	Only own claims show; filter narrows by policy type.	[ ] Pass
TC-09	State sharing	Create claims for CA and TX policies.	Policy Account Holder State is filled; sharing rules apply.	[ ] Pass
TC-10	Permission sets	Log in as an agent, adjuster and manager.	Each user has only the access of their permission set.	[ ] Pass
TC-11	Apex test	Run ClaimsAdjusterControllerTest.	Test passes with 95% or more coverage.	[ ] Pass

06

Plan and Schedule

Agile sprints and story points

We followed an Agile approach: work was split into epics and user stories, each sized in story points, and the team velocity was used to check that we stayed on track. The work was done in four sprints.

Sprint	Epic	Story	User story / task	Pts	Priority	Owner
1	Data Modeling	US-01	As an admin, I want to create Policy and Claim objects so that policy and claim information can be stored and managed systematically.	5	High	S. Nithish
1	Data Modeling	US-02	As an admin, I want to configure Vehicle, Property and Life Record Types so that different products can be managed in one Policy structure.	3	High	G. Anish Pradeep
1	Data Modeling	US-03	As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance information is captured efficiently.	5	High	S. Johnson Infant
1	Quoting and Automation	US-04	As an insurance agent, I want a guided Auto Quoting Screen Flow so that policy information is captured in a standard way.	8	High	S. Bharathraj
2	Validation	US-05	As an insurance agent, I want VIN validation during vehicle quoting so that incorrect vehicle data is prevented.	3	High	S. Nithish
2	Apex	US-06	As an insurance system, I want PremiumCalculator Apex to calculate premiums so that premium values are accurate.	8	High	G. Anish Pradeep
2	Automation	US-07	As an insurance agent, I want the premium calculation integrated with the quoting Flow so that the premium is available during the quote.	5	High	S. Johnson Infant
2	Claims Automation	US-08	As a claims manager, I want claims routed by policy type so that they reach the right support queue.	8	High	S. Bharathraj
3	Apex	US-09	As a claims adjuster, I want a ClaimsAdjusterController to retrieve my assigned claims so that I can manage my workload.	8	High	S. Nithish

Sprint	Epic	Story	User story / task	Pts	Priority	Owner
3	LWC Dashboard	US-10	As a claims adjuster, I want a claims dashboard and reusable claim tile so that I can view and manage assigned claims.	8	High	G. Anish Pradeep
4	Approval	US-11	As a claims manager, I want high-value claims to go through an approval process and approver Flow so that they get proper review.	8	High	S. Johnson Infant
4	Security and Testing	US-12	As a system administrator, I want tests, sharing rules and Permission Sets so that the solution is validated and users get the right access.	8	High	S. Bharathraj

Sprint summary and velocity

Sprint	Planned points	Completed points	Stories	Duration
Sprint 1	21	21	US-01 to US-04	6 days
Sprint 2	24	24	US-05 to US-08	6 days
Sprint 3	16	16	US-09, US-10	6 days
Sprint 4	16	16	US-11, US-12	6 days
Total	**77**	**77**	**12 user stories**	**24 days**

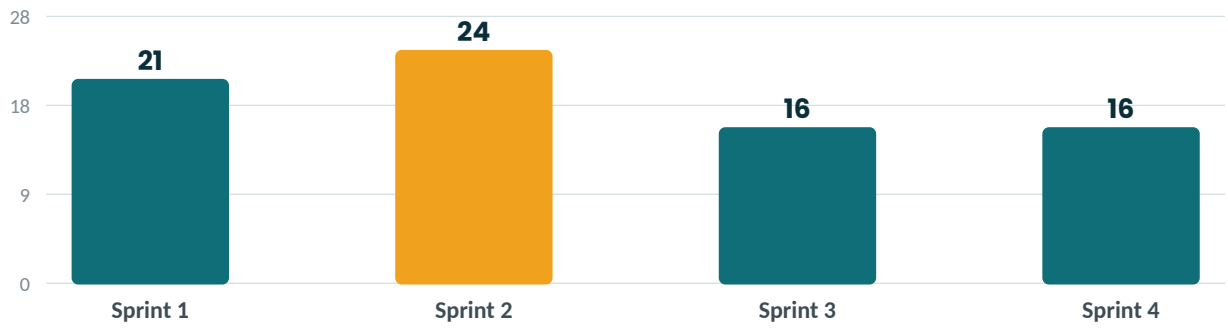


Figure D. *Story points per sprint (total 77, peak load in Sprint 2 shown in amber).*

Velocity

77 story points over 4 sprints is about 19.3 points per sprint. Sprint 2 carried the most work because the PremiumCalculator Apex logic, its Flow integration and the claim routing flow were built together.

07 Strengths and Limits

An honest look at what we built

STRENGTHS

- **Centralized platform:** Vehicle, Property and Life policies and claims in one Salesforce system.
- **Automated quoting:** Screen Flows simplify and standardize quoting.
- **Automatic premiums:** Apex calculates premiums the same way every time.
- **Claim routing:** claims reach the right queue by policy type.
- **Faster handling:** the approval process reduces manual effort for high-value claims.
- **Better visibility:** the LWC dashboard gives adjusters a focused view.
- **Accurate data:** validation rules keep policy information correct.
- **Flexible model:** Record Types and Field Sets fit different products.
- **Secure access:** Permission Sets and sharing rules give role and state-based access.
- **Scalable:** the design can take more lines and features.

LIMITS

- **Complex data:** Vehicle, Property and Life policies need different fields to manage.
- **Approval complexity:** approval must consider claim amount, policy type and adjuster level.
- **User adoption:** agents need intuitive flows with as few clicks as possible.
- **Integration effort:** simulating external database callouts needs more Apex and mock test classes.
- **Platform dependency:** the solution depends on Salesforce configuration and capabilities.
- **Implementation effort:** many Flows, Apex classes, LWC components, security rules and approvals need careful development and upkeep.
- **Single threshold:** the $50,000 approval limit and queue Ids are set by hand in the flows.
- **Developer Edition only:** built for academic purposes and not deployed to production.
- **Deployment tools:** a real rollout needs sandboxes, Change Sets or Salesforce DX.

08 Conclusion and Roadmap

What we learned and where we would take it next

The Multi-Line Insurance Policy and Claims Management System shows how a standard Salesforce org can be shaped into a working policy and claims platform. Vehicle, Property and Life insurance data now lives in one place. Guided quoting, automatic premium calculation, claim routing, structured approvals and an adjuster dashboard replace the manual and fragmented steps of the old process.

The solution combines Flows, Apex, Record Types, Field Sets, Validation Rules, Approval Processes, Lightning Web Components, Permission Sets and Sharing Rules to improve policy processing, claim handling, data accuracy, security and operational visibility. Through this project the team learned data modelling, Screen and Record-Triggered Flows, Apex and test classes, Lightning Web Components and the Salesforce security model. Overall, the project demonstrates how Salesforce can bring multiple insurance processes together on one scalable platform, helping insurance teams work efficiently and serve customers better.

Where we would take it next

Area	Next step
External integration	Connect to external vehicle and property databases for real-time quote verification.
More insurance products	Extend the solution to lines beyond Vehicle, Property and Life.
Advanced claims automation	Add automated claim assessment and escalation based on claim characteristics.
Enhanced analytics	Build advanced dashboards and reports for policy, premium and claims analysis.
Customer experience	Add customer-facing tracking of policy and claim status.
Mobile access	Let agents and claims users reach key processes from mobile devices.
AI assistance	Explore AI help for agents and adjusters with policy and claim tasks.
Integration architecture	Expand API-based integrations with insurance and verification systems.

09 Deployment and Maintenance

Taking the build from a Developer org to production, and keeping it healthy

The system was built and tested in a Developer Edition org for academic purposes. This section explains how the same configuration and code could be promoted to a real insurance business, how it would be looked after once live, and a short list of technical improvements spotted while documenting the build.

Deployment path

Stage	Environment	What happens
1. Build	Developer Edition	Objects, flows, Apex, LWC and permission sets built and unit-tested (this project).
2. Package	Source control	Retrieve the metadata with Salesforce DX, or add the components to an outbound Change Set, and keep it in Git.
3. Integrate	Developer / Partial sandbox	Deploy, load sample customers and policies, then run test cases TC-01 to TC-11.
4. Accept	Full sandbox (UAT)	Agents, adjusters and managers try real scenarios; fixes are logged and retested.
5. Release	Production	Run a validate-only deployment first, then deploy, activate the flows, assign permission sets and confirm the queues.
6. Support	Production	Hypercare for the first weeks, then the maintenance plan below.

Pre-release checklist

Item	Check before go-live
Queues	Queue Ids in the routing flow are re-pointed to the production queues, or the queues are looked up by name.
Approvers	The Senior Adjuster and Claims Manager users named in the approval steps exist and are active.
Access	Insurance Agent, Claims Adjuster and Claims Manager permission sets are assigned; CA and TX sharing is checked with a test user.
Flows	Every flow is Active and has a fault path that emails an administrator.
Tests	All Apex tests pass and org-wide coverage is at least 75%, the minimum Salesforce requires to deploy to production.
Data	Policy State picklist values (CA, TX) match the codes used in PremiumCalculator.

Maintenance plan

Task	How often	Owner
Review failed flow interviews and error emails	Weekly	System Administrator
Run all Apex tests and check coverage	Before every release	Developer
Re-check premium rates, the model-year cut-off and the $50,000 threshold	Every quarter	Claims Manager
Audit permission sets and sharing rules for leavers and role changes	Every quarter	System Administrator
Read Salesforce release notes for changes that affect flows, Apex and LWC	Three times a year	System Administrator
Refresh sandboxes and re-run regression tests after major changes	As needed	Developer

Suggested technical improvements

Area	What we noticed	Suggested change
Days Open	ClaimsAdjusterController works out Date.today().daysBetween(Date of Loss), which is negative for a loss date in the past.	Reverse it: Date of Loss.date().daysBetween(Date.today()).
Policy type	The query selects Policy__r.RecordType.DeveloperName, but the wrapper reads RecordType.Name, which was not queried.	Select and read the same field, then re-run ClaimsAdjusterControllerTest.
Premium results	PremiumCalculator returns values in query order rather than input order, and fails on a non-numeric model year.	Key results by policy Id, return them in input order and guard the Integer conversion.
Test coverage	Only the dashboard controller has a test class.	Add PremiumCalculatorTest for CA, TX, other states and the pre-2018 surcharge.
Hard-coded values	Queue Ids, the $50,000 limit and the premium rates are typed into flows and Apex.	Move them to Custom Metadata and look queues up by developer name.
State copy flow	Claim Policy Holder State Update loops over every policy returned by Get Records.	Read the state from the claim's own Policy lookup so each claim gets its own policy state.
Dashboard targets	The text says Record page, but the meta file exposes only App and Home pages.	Add lightning__RecordPage to the targets if tiles are needed on record pages.

These points do not change the scope of the project; they are suggestions for the next iteration.

A Appendix

Reference items and team

Ref	Item	Description
A	Custom objects	Policy__c, Claim__c (with Contact and User lookups)
B	Record Types	Policy: Auto, Property, Life. Claim: Accident, Property, Life
C	Automation	AutoQuotingFlow (Screen), claim routing flow, Submission Automation Flow, Claim Approver Screen Flow, Claim Policy Holder State Update flow; High Value Claim Approval process
D	Apex	PremiumCalculator, ClaimsAdjusterController
E	LWC	claimsDashboardLwc (Claims Dashboard), claimTileLwc (Claim Tile)
F	Validation	VIN_Must_Be_17_Characters on Policy
G	Security	Permission sets Insurance Agent Access, Claims Adjuster Access, Claims Manager Access; state-based sharing rules (CA, TX)
H	Testing	ClaimsAdjusterControllerTest
I	Reporting	Claims Dashboard and Salesforce Reports and Dashboards

Project team

Team member	Role	NM ID	Register No.
S. Nithish	Team Leader	`13B27FAE19C56F09F3543B6453644851`	`821923243037`
G. Anish Pradeep	Team Member	`E588806BA58D395989045D185F8FC289`	`821923243007`
S. Johnson Infant	Team Member	`B79E6A8BC3205E4B2EA407B0225A3E24`	`821923243023`
S. Bharathraj	Team Member	`55E53093039601EBC7E49C15B5D3CF60`	`821923243012`

St. Joseph's College of Engineering and Technology, Thanjavur | College Code: 8219 | Team ID: SWTID-2026-8095